Electronics and
Computer Science

UNIVERSITY OF
Southampton

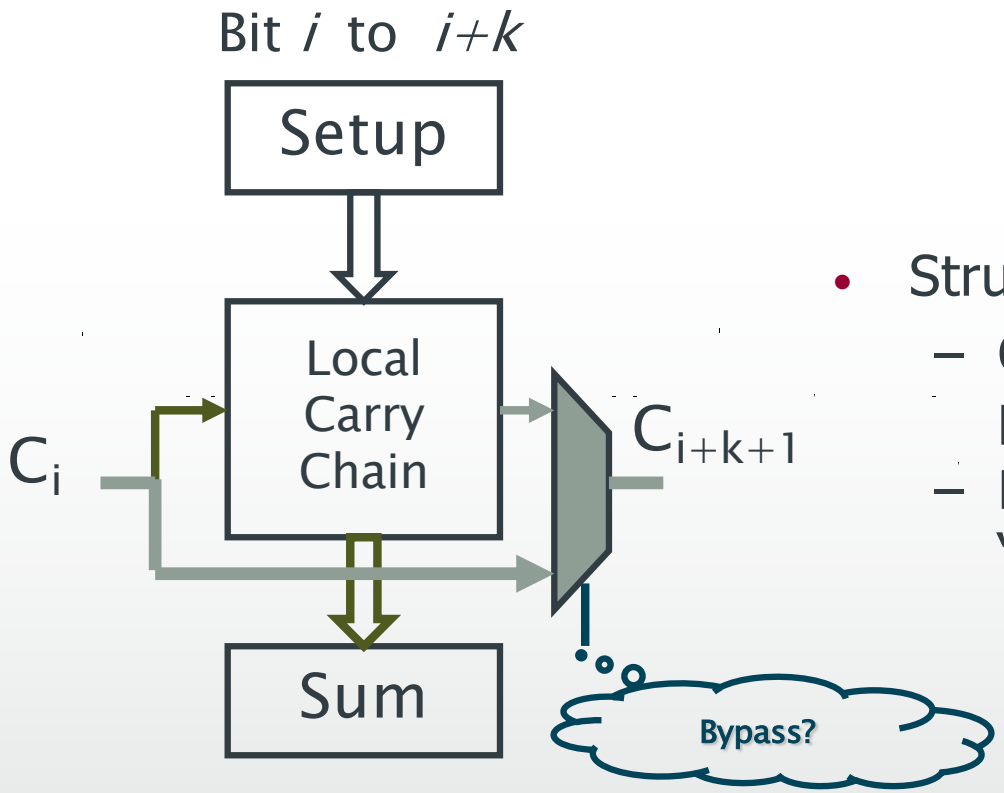
13. Computer Arithmetic – Adders and Subtractors

Manchester Carry Chain

- Reuses P, G logic for intermediate carry signals
  - Not suitable for CMOS gates
  - Transistor level (pre-charge internal nodes)
  - Good for up to 16 bits

Carry-bypass adder

- Idea is to bypass carry logic using P signals
- The local carry chain could be “ripple carry adder”



- Structure
  - Could be static, dynamic, pass transistor
  - Bypass logic determines: “pass” or “kill/generate”?

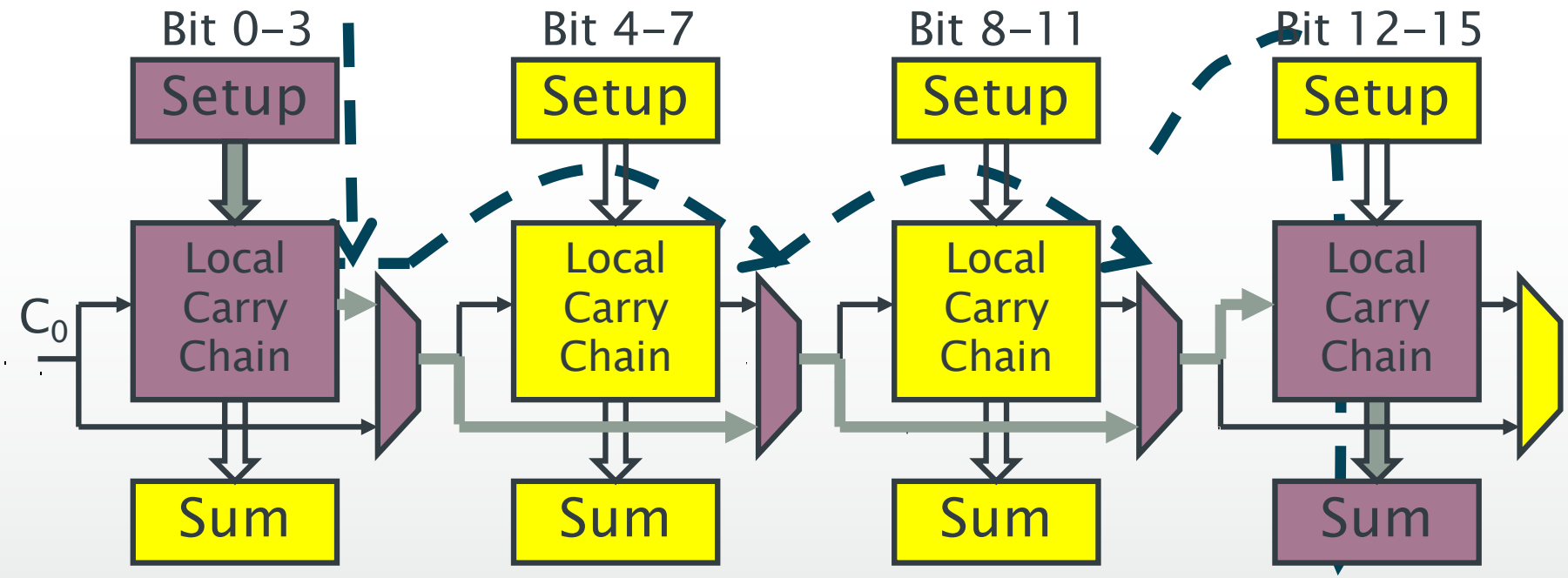
Carry-bypass adder: structure

Timing (Critical path shown in different color):

1-Setup

2-Local carry generate/kill, MUX select line ready

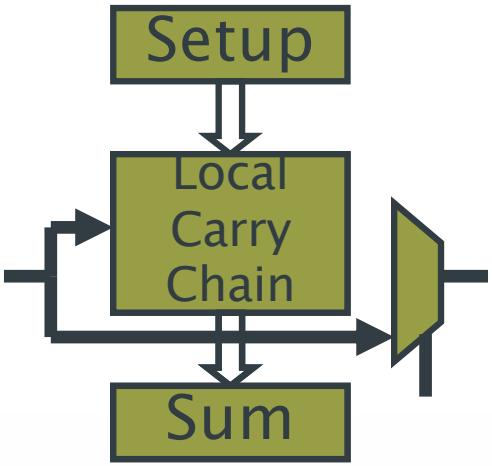
3-C_0-C_{16} carry propagate (if applicable)

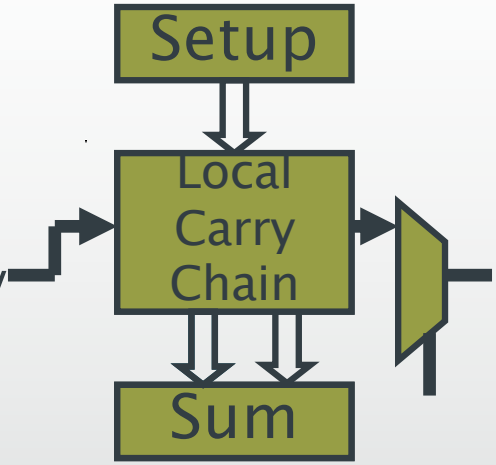


Carry-bypass adder: timing of sub-block

For an intermediate stage, after setup:

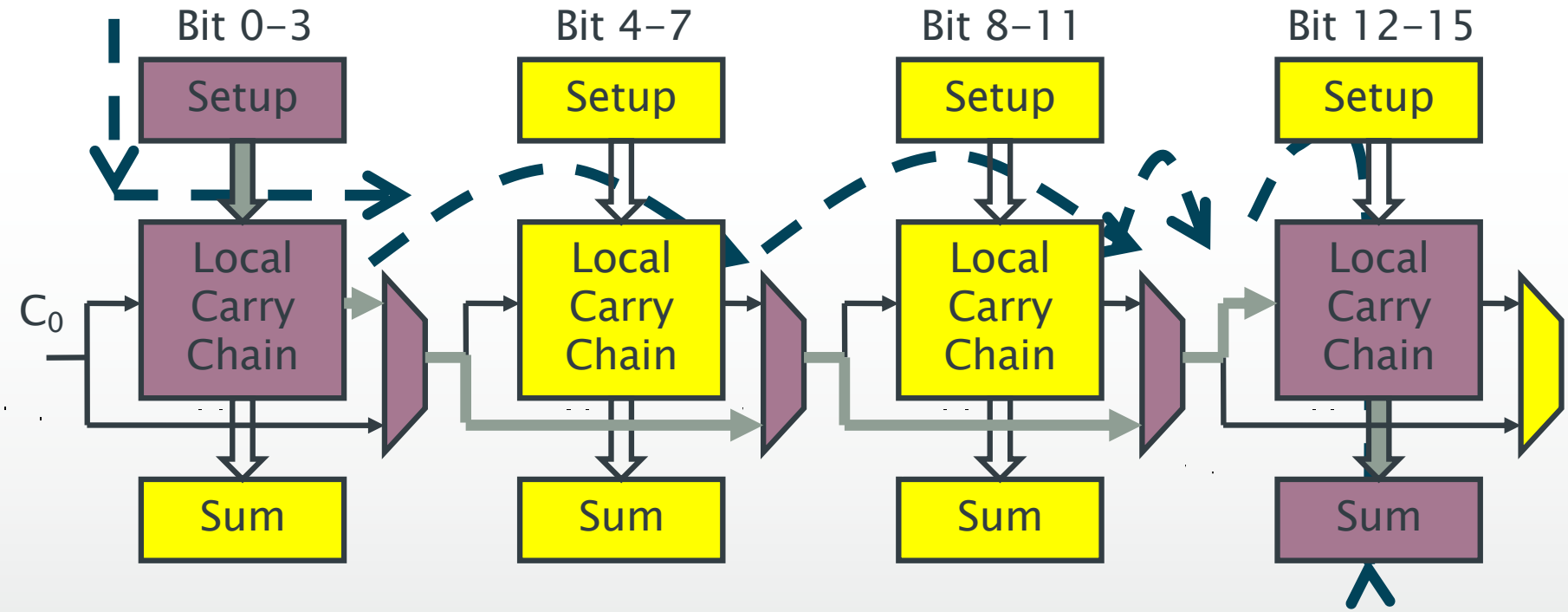
- If in *pass* mode
  - Local carry vector computes intermediate carries (possibly incorrectly)
  - At the same time, mux selection set to pass
  - When input carry arrives, intermediate carries might be recomputed
  - Meanwhile, input carry is sent to Cout
- If *not pass* mode (assume bit 10 generates)
  - Local carry vector computes intermediate carries (bits 10, 11 correct)
  - At the same time, mux selection set to local
  - Meanwhile, output carry is sent to Cout correctly
  - When input carry arrives, intermediate carries C_8 and C_9 (S_8,S_9,S_{10}) will be recomputed correctly





Carry-bypass adder: timing

$$\text{Delay} = t_{setup} + \max\{t_{select}, 4 \times t_{FA}\} + 3 \times t_{mux\_pass} + 4 \times t_{FA}$$



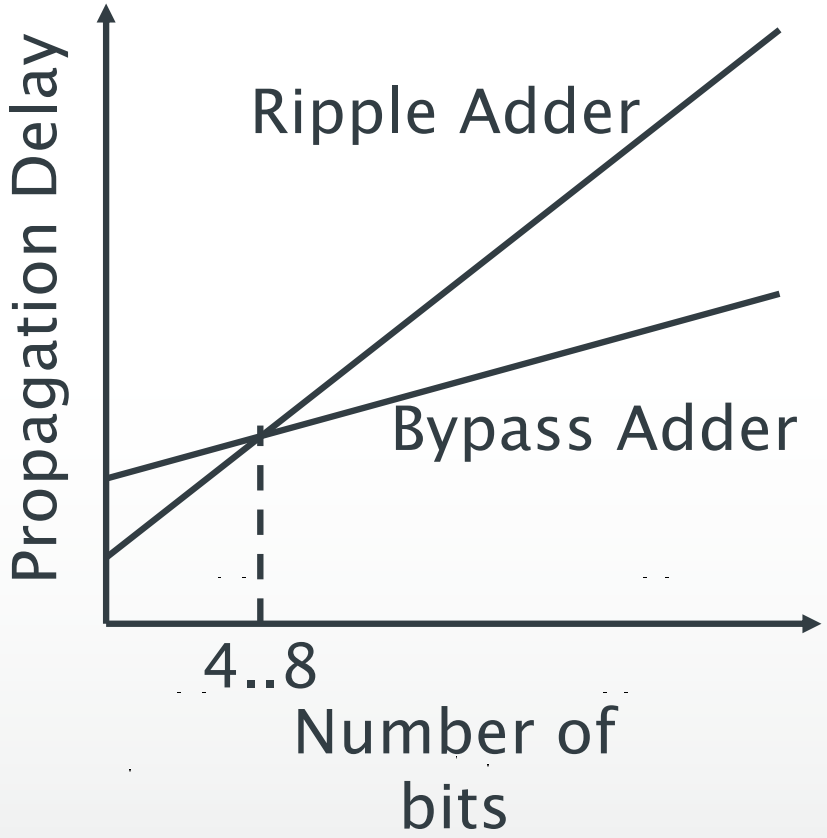
Carry-bypass adder: pros and cons

Speed:

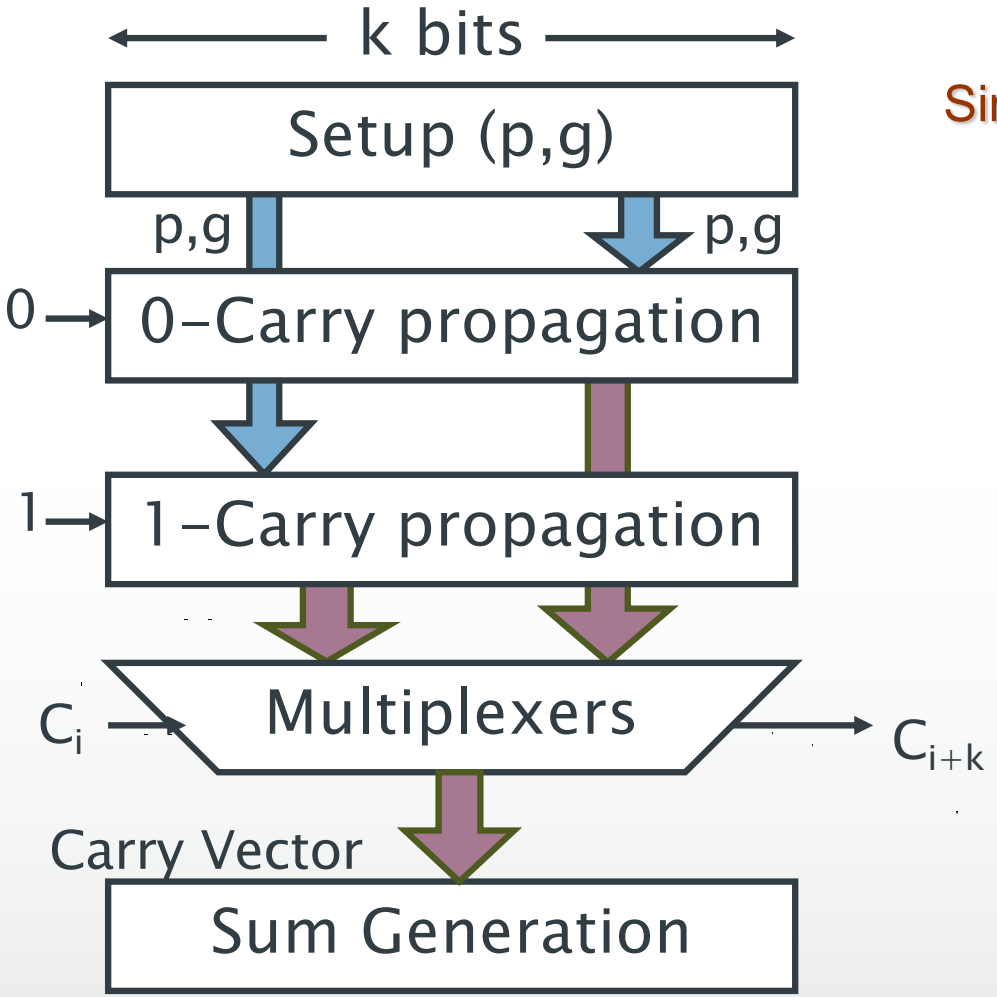
- Faster than ripple adder
- Still linear!

Area overhead:

- Mux (setup?)
- Not worth for small adders (N<8)
- 10-20% for large adders



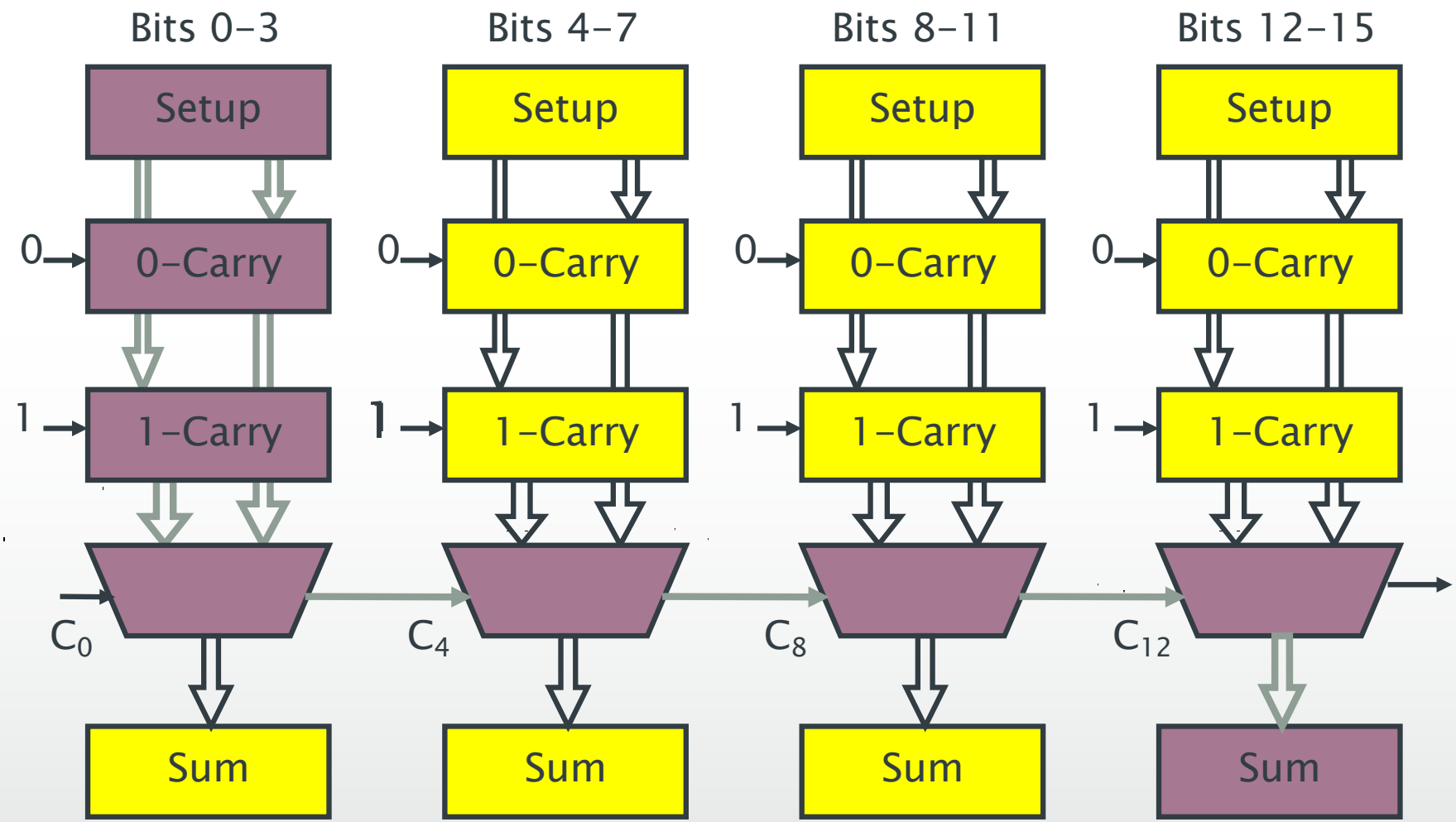
Carry-select adder: idea



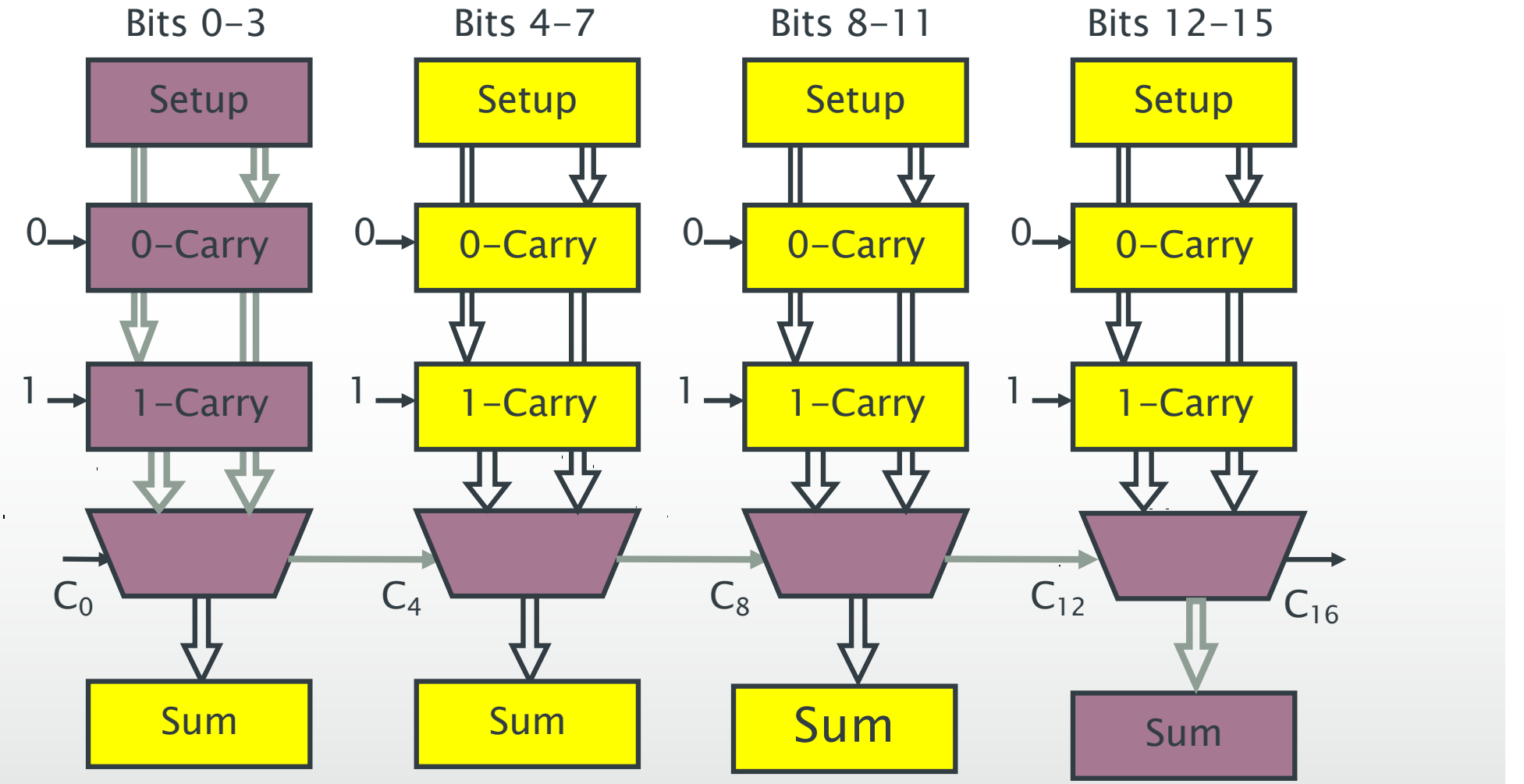
Similar to bypass

- Instead of "waiting" for the input carry, "precompute" the carry output
- Compute C_{i+k} for both cases $C_i=0$ and $C_i=1$
- When C_i arrives, select the appropriate result
- Sum computed in one step after the intermediate carry signals are ready

Linear carry-select adder: structure



Linear carry-select adder: timing



Delay = 3 + 1 + 1 + 1 + 1 = 7 (16 bits)

Square root carry select adder: idea

Later stages have to wait for the multiplexers in the earlier stages

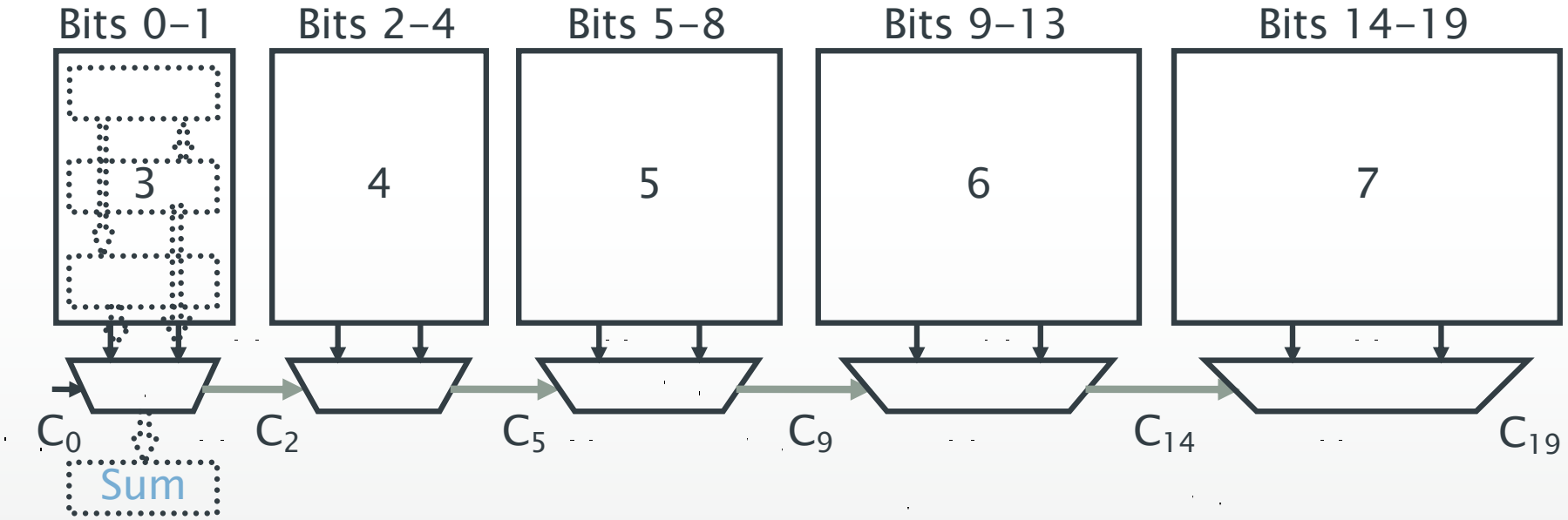
Why not give them bigger chunks of data to compute?

- Balances the delay paths
- Sub-linear delay

Square root carry select adder: structure

Assuming the following delays:

- Setup=1, carry propagate=1/bit, mux=1



Delay from all paths = 8 (20 bits)

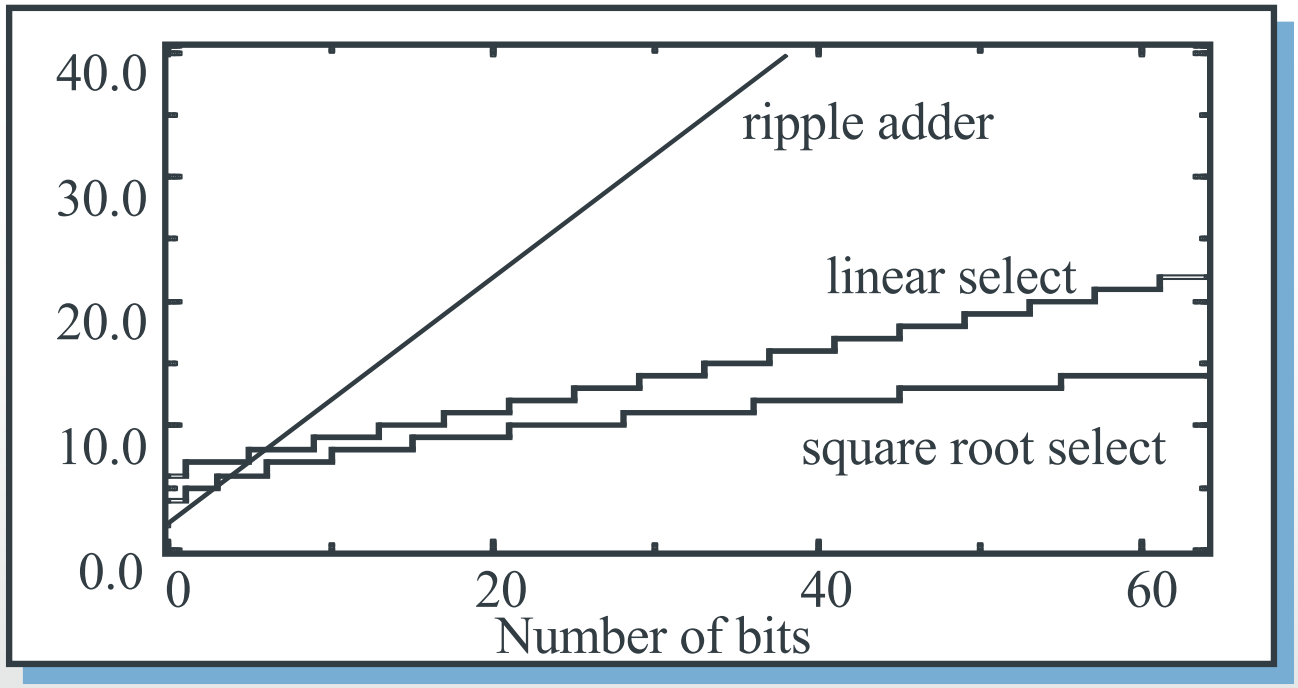
Carry select adder: trade-offs

Area overhead:

- An additional carry path and a multiplexer (not the whole adder)
- About 30% more than a ripple-carry

Delay

- Sub-linear (we can beat that too!)



Crude adder speedup

Ripple (Slowest)

Carry-Lookahead

Carry-Select

Manchester

Transmission Gate (Faster)

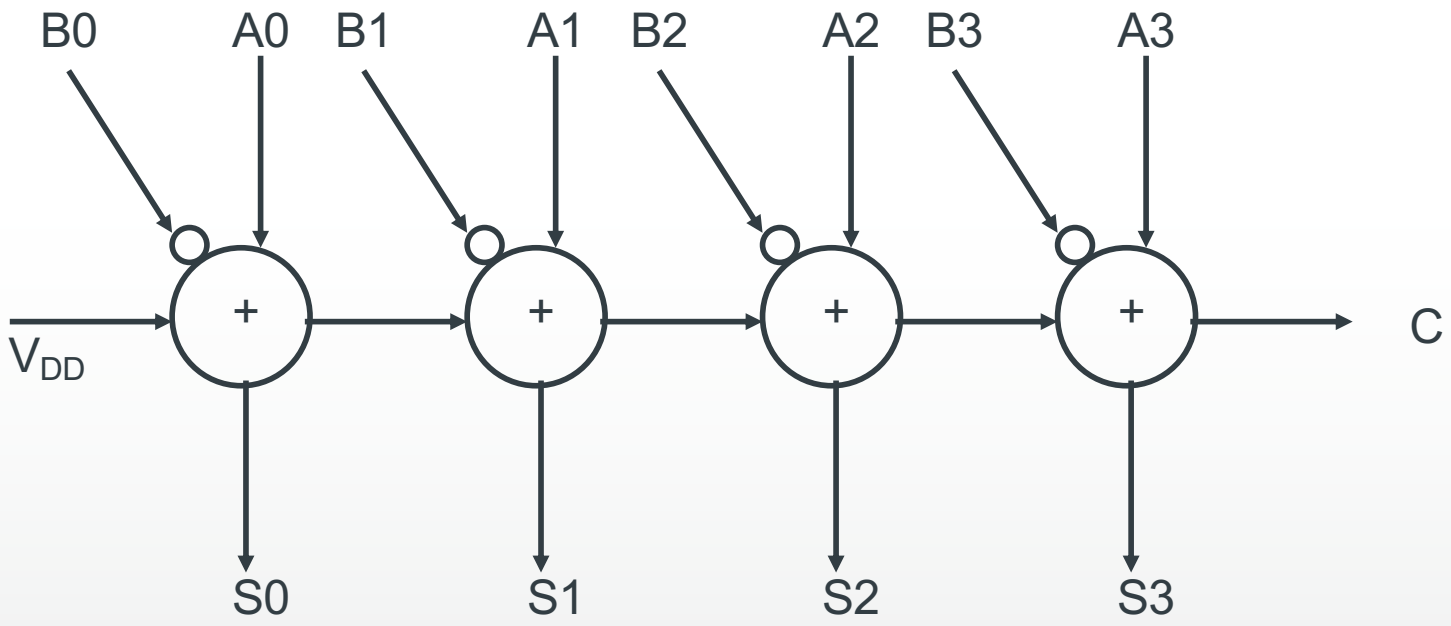
Block Carry-Lookahead

Use block carry lookahead for floating-point ALU's

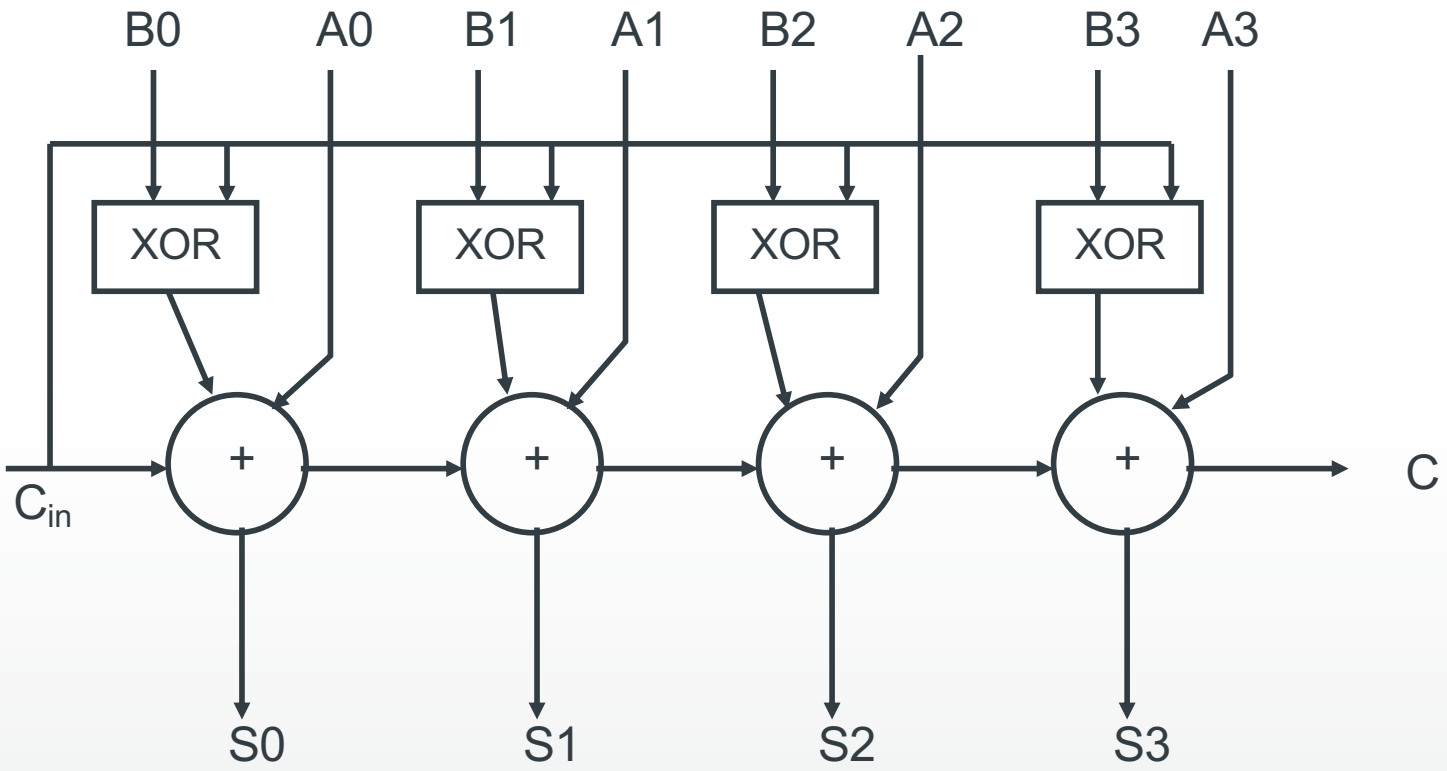
Subtractor

- Subtractors are the same as adders in construction
- Subtraction is normally done using 2's complement numbers
  - The subtrahend has to be converted to 2's complement form
  - Add the 2's complemented subtrahend to the minuend

Subtractor circuit



Adder/Subtractor circuit



If C_{in} = 0, the circuit will act as adder, If C_{in} = 1, the circuit will act as subtractor

Carry logic

A	*B*	*Cin*	*Sum*	*Cout*
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

When coding addition as A+B,
we don't know Cin at MSB, but we know Sum[7]

Carry is set at MSB if:

A[7] & B[7] | B[7] & ~Sum[7] | A[7] & ~Sum[7]

In subtraction A-B, complement B[7]

Detection of 2's complement overflow

A	*B*	*Cin*	*Sum*	*Cout*
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Overflow occurs in addition if the signs of both operands are the same, but the result sign is different, i.e.

A[7] & B[7] & ~Sum[7] |
~A[7] & ~B[7] & Sum[7]

In subtraction B-A, invert A[7]

Some processors (MIPS, Arm) set Carry and Overflow flags

```
RADD : begin
  V = a[7] & b[7] & ~sum[7] | ~a[7] & ~b[7] & sum[7];
  C = a[7] & b[7] |  a[7] & ~sum[7] | b[7] & ~sum[7];
  end
RSUB : begin
  V = a[7] & ~b[7] & ~sum[7] | ~a[7] & b[7] & sum[7];
  C= a[7] & ~b[7] | a[7] & sum[7] | ~b[7] & sum[7];
  end
```